

# Proposed solution

## 1. Introduction

The primary objective of this project is to conduct an in-depth study of the Linux scheduler and its different scheduling policies. The project is divided into two main parts;

- Investigating the mapping between Java (JVM) threads and Linux native threads. We will examine how Java thread priorities are mapped to native Linux processes and priorities. For example, we will look into how Java platform threads are typically mapped 1:1 to kernel threads scheduled by the operating system.
- Designing and implementing a timing-critical periodic controller in C and evaluating its performance using different Linux scheduling policies, with a special emphasis on SCHED\_DEADLINE. We will look into latency, jitter, and deadline misses under varying system loads using native and/or custom load generators.

The hardware that we will use is a Linux PC to be able to run both Java threads and Linux Native threads, and utilize the PC's CPU cores to pin the controller to a specific core using CPU affinity. This will be done using tools such as `app-rt` and `stress-ng`. To test the mapping between Java and Linux threads we will use a simple program written in Java. The periodic controller will be written in C. To test the controller we will use the DC Servo Motor used in lab 3. To control the motor we will not use the Atmega as in Lab 3, but an external I/O library called `Moberg`. To tune the controller parameters in real time we will use a simple UI program written in Python.

## 2. Program Structure

To investigate and compare Java threads and Linux threads we will utilize the Java Thread class and the native Linux threads library (nptl). For implementing the simple periodic controller we will use C and it will be run using different native Linux scheduling policies: `SCHED_OTHER`, `SCHED_OTHER`, `SCHED_OTHER` and `SCHED_DEADLINE`. To ensure the results are as accurate as possible we will use CPU affinity and pin the controller to a specific core in the CPU. An initial assessment of pinning the controller to a specific core is using the Linux `sched_setaffinity` and `taskset`. To measure the performance of the processes tools like `perf` and `strace` can be used from Linux own process library, we can also measure the time using Linux own tools, we can use an oscilloscope by outputting a high volt when we start and end the process and then measure the time it takes from the two high voltages or we use a timing measuring register to measure the execution time in clock cycles instead of comparing the start and end time of a code block. To plot the data and tune the controller in real time (by changing the constants) we can use a pipeline and plot the data and control constants using the Unix sockets as the pipeline between the controller C program and the UI Python program.

To investigate the Java and Linux thread mapping we will set priorities of the Java threads and investigate how they are scheduled to the Linux threads. To monitor the threads and get output data we can use the Java Threads stack traces and prints together with Linux tools such as `Scheduler debugs` and `Scheduler statistics`. Linux commands such as `top`, `stop` and `ps` can be used to see how the JVM threads appear in the Linux kernel as PIDs.

### 3. Operator Communication

To communicate with the controller using our operator we will initially use a terminal-based interface before we implement the Python UI program to make sure everything works. Since the main purpose of the project is to investigate scheduling behavior, a more advanced graphical interface is likely not needed but it will be implemented since it will become easier to tune controller parameters and visualize system and control outputs, reference signal and so on. The operator should be able to start the controller with different parameters such as scheduling policy, period, runtime, deadline and CPU cores.

For the Java thread investigation, the operator will mainly run test programs and inspect the results using Linux tools. During the execution, the program can measure timing information and basic statistics using Python, or as described in section 2, using the measuring register. The operator can both configure the experiments and study the timing behavior of the Java program.

### 4. Control Principles

We will take inspiration from lab 3 and control the angular velocity using a simple PI controller. It will be designed by determining the constants of the equation  $c(s) = K(1 + 1/sT_i)$  and placing closed loop poles at desired spots. For simplicity we will just have a reference generator at 5 V as in Lab 3.

## 5. Project Plan

Our initial project plan is that we work two times a week and have meetings, if we get close to the deadline and do not have much results we will put in extra days. At the start we will research all the information that we need, and as soon as possible we will start building the project. We will go through the objectives 1-5 one at a time, starting with the Java thread investigation. Once we are done with that, we will move on to implementing the controller and testing it using the DC-motor. When the controller is working, we will implement the operator and start measuring behaviour of the different scheduling policies.